

BoardScan

Chess Position Digitiser

(Photo → FEN → Evaluation)

Submitted by

Abhishek Kushwah

Reg. No.: 24BAI10540

TABLE OF CONTENTS

1. Introduction.....	2
2. Problem Statement.....	2
3. Functional Requirements.....	2
4. Non-Functional Requirements.....	3
5. System Architecture.....	3
6. Design Diagrams.....	4
7. Design Decisions and Rationale.....	5
8. Implementation Details.....	6
9. Results.....	7
10. Testing Approach.....	9
11. Challenges Faced.....	10
12. Learnings and Key Takeaways.....	10
13. Future Enhancements.....	10
14. References.....	11

1. Introduction

Players who play over-the-board chess usually have to type a position into an analysis engine by hand if they want to study it later. This is slow and it is easy to make mistakes, especially when the position has many pieces. BoardScan is a tool that takes a single photograph of a physical chessboard, taken from any reasonable angle, and returns the position in Forsyth–Edwards Notation (FEN) together with an engine evaluation.

The board is located using classical computer-vision techniques only: edge detection, the Hough transform and a perspective homography. A small convolutional neural network (CNN) is used for just one job, which is classifying the piece on each of the 64 squares. This report covers the requirements, design, implementation and evaluation of the system, and discusses the results honestly, including the places where the system does not yet meet its targets.

Source code: <https://github.com/abhishek-k-33/boardscan>

2. Problem Statement

Manually re-entering over-the-board positions into analysis engines is slow and error-prone. The aim of this project is to automate that step.

2.1 Target Users

- **Club players** who want to digitise their games for later analysis.
- **Coaches** who review positions from their students' games.
- **Tournament arbiters** who need to record positions quickly and accurately.

2.2 Scope

The system accepts one RGB photograph of a complete board, taken with a tilt of 0° to 45° and under varied lighting, and produces a validated FEN string and an engine evaluation.

Out of scope: video tracking, partially visible boards, and OCR of clocks or scoresheets.

3. Functional Requirements

The system is divided into four modules. Each module has a clearly defined input and output so that it can be developed and tested on its own.

Table 1: Functional modules

No.	Module	Input	Output
1	Detection and Rectification	RGB photo	800×800 top-down board image
2	Square Segmentation and Classification	Warped board	8×8 label grid (13 classes)
3	FEN Assembly and Engine Analysis	Label grid + side to move	FEN, centipawn evaluation, best move
4	Debug / Visualisation Layer	Output of any stage	Annotated overlay images

4. Non-Functional Requirements

Table 2: Non-functional requirements

Requirement	Target / Evidence
Performance	Full CLI pipeline runs in 0.07–0.08 s on CPU against a budget of 3 s; input is downscaled to 1200 px.
Reliability	Every stage returns a typed ok / data / error result, so the UI can show an error message instead of crashing.
Usability	The 8×8 grid is editable, so a single wrong square never forces the user to retake the photo.
Maintainability	All CV parameters live in config.py; the code is made of pure functions that can be tested independently.
Observability	The --debug flag writes numbered images of every stage to the debug/ folder.
Resource efficiency	The CNN has about 212k parameters and is 829 KB in fp32 (budget 5 MB); it runs on CPU only.

5. System Architecture

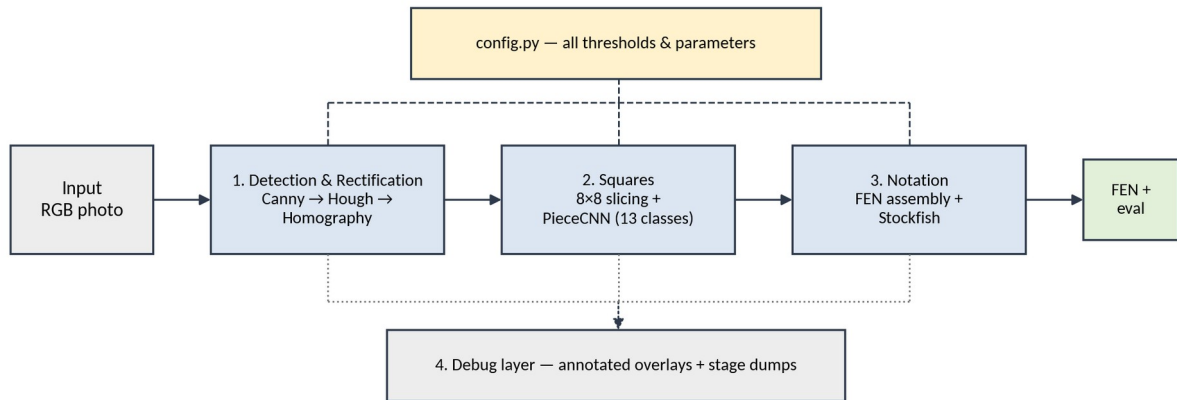


Figure 1: System architecture – module data flow from photo to FEN and evaluation

The processing pipeline works in the following order. The photo is first preprocessed (grayscale conversion, CLAHE and a bilateral filter). Canny edge detection and the probabilistic Hough transform then find the grid lines of the board. The four board corners are estimated in two ways, from the line lattice and from a contour fallback, and the method that gives the larger quadrilateral is used. A homography warps the board into an 800×800 top-down image, which is sliced into an 8×8 grid of squares (with a small upward bias so that tall pieces are not cut off). Each square is classified by PieceCNN, and the labels are assembled into an orientation-aware FEN string with king-count validation. Finally, the FEN is analysed by Stockfish, or, if Stockfish is unavailable, the system falls back to a degraded FEN-only mode.

config.py supplies parameters to every stage, and the debug layer can tap into each stage to save intermediate images. Storage is file-based: raw photos are kept in data/raw with .fen sidecar files, and square crops are kept in data/squares along with manifest files. Since no database is used, an ER diagram is not applicable to this project.

6. Design Diagrams

6.1 Use Case Diagram

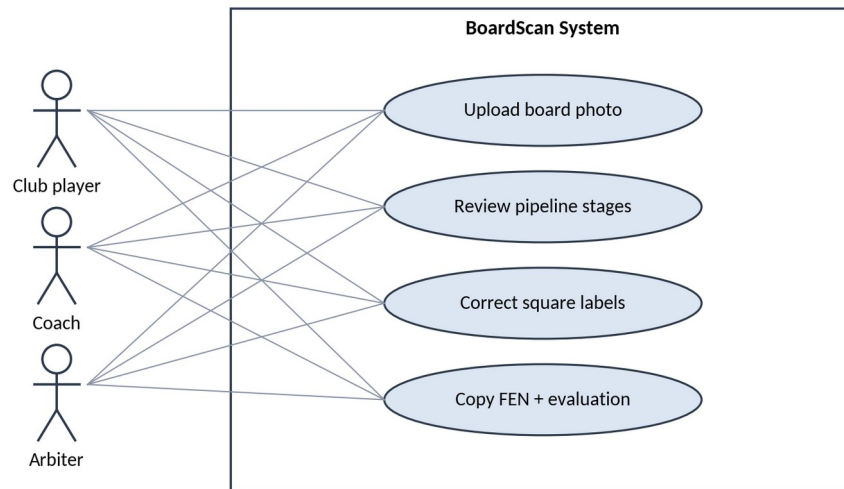


Figure 2: Use case diagram

All three types of users interact with the system in the same four ways: they upload a photo of the board, review the output of each pipeline stage, correct any wrongly labelled squares, and copy the final FEN and evaluation.

6.2 Sequence Diagram

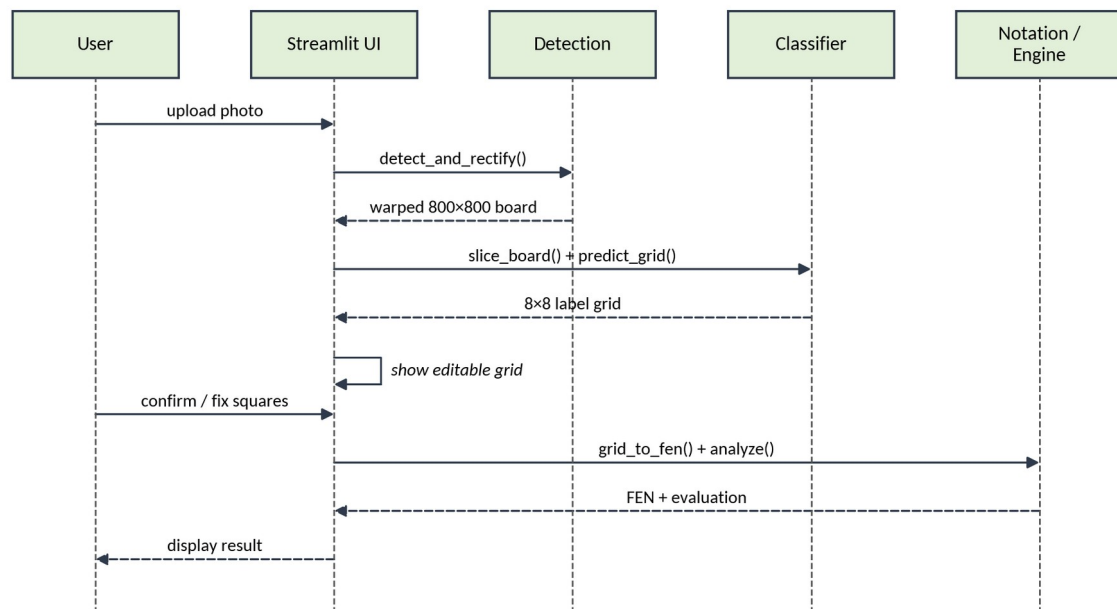


Figure 3: Sequence diagram – upload to FEN

The Streamlit UI coordinates the other modules. After classification the user is shown an editable grid, and only after the user confirms or fixes the squares is the FEN built and sent to the engine.

6.3 Class Diagram

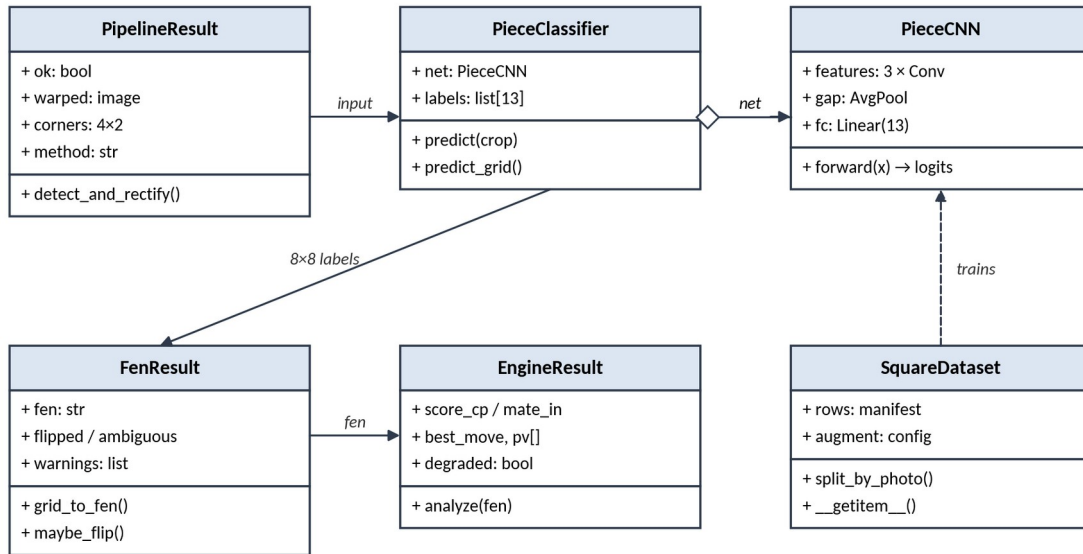


Figure 4: Class diagram – key result containers and their relations

The workflow, in pipeline order, is: upload → detect_and_rectify → slice_board → predict_grid → grid_to_fen → analyze → display. Each step passes a typed result to the next, and any failure short-circuits to a message that is shown to the user.

7. Design Decisions and Rationale

Table 3: Design decisions

Decision	Rationale
Bilateral filter instead of Gaussian	Preserves the board edges while smoothing out wood grain. It can be switched with a single flag for ablation.
CLAHE instead of global histogram equalisation	Handles uneven lamp light falling across the board.
Probabilistic Hough instead of standard Hough	Segment endpoints make angle clustering possible; <code>minLineLength</code> is tied to the expected square size.
Lattice + contour fallback (larger quad wins)	The lattice can extrapolate corners that are clipped, while the contour can trace borders that are partly occluded.
Per-square CNN instead of whole-board detector	64 easy 13-class problems are more practical than one hard 32-object detection problem when only about 8,000 training squares are available.
Split by photo, not by square	Squares from the same photo share lighting, so a naive split inflates accuracy. We reproduced a fake 100% this way.
King-count validation	More than one king per side is treated as an error; missing kings (e.g. empty calibration boards) only produce a warning.
Mixed synthetic + realistic training data	Training on a single style collapses on unseen board art (train/serve skew).

8. Implementation Details

8.1 Technology Stack

Python, OpenCV primitives, NumPy, PyTorch (used for the classifier only), python-chess, Streamlit and pytest.

8.2 Code Structure

Table 4: Package layout

Package	Contents
detection/	preprocess.py, lines.py, corners.py, rectify.py, pipeline.py – pure, config-driven functions.
squares/	slicer.py (100 × 100 crops with a +25 px upward bias for piece heads), labels.py (FEN ↔ grid), classifier.py.
model/	PieceCNN (48/96/192 channels + global average pooling + FC(13)); SquareDataset with brightness/contrast, ±3° rotation and ±2 px shift augmentation. Horizontal flip is not used.
notation/	fen.py (orientation score, auto-flip, ambiguous flag) and engine.py (fixed-depth Stockfish with a degraded mode).

8.3 Training and Data

- The training set has 8,572 squares: 3,900 synthetic and 4,672 taken from rendered boards.
- Data is split 70 / 15 / 15 by photo. The model is trained with Adam (learning rate 1e-3) and a StepLR schedule on 96 px inputs.
- Board images come from a sample of the honi05/chess-positions-cv dataset, where the FEN is stored in the file name. They were imported as data/raw/NNN.jpg with a matching NNN.fen file. The use of this dataset is disclosed openly.

9. Results

9.1 Pipeline Stages

The images below are the debug dumps produced for data/raw/000.jpg.

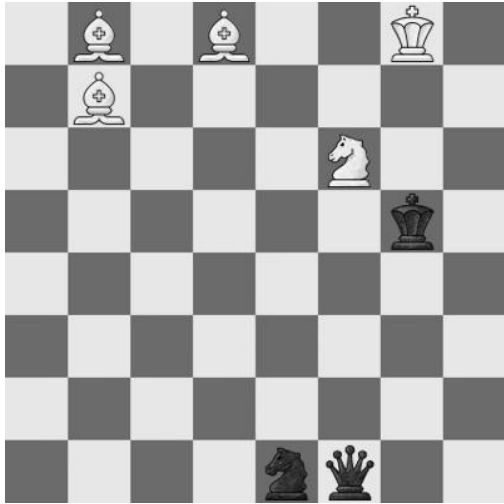


Figure 5: Grayscale + downscale

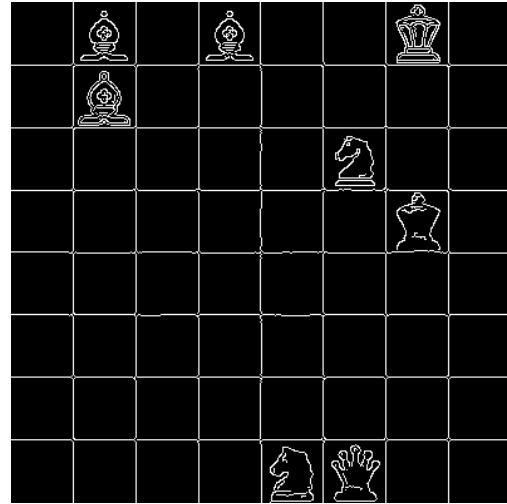


Figure 6: Canny edge map



Figure 7: Rectified 800×800 board

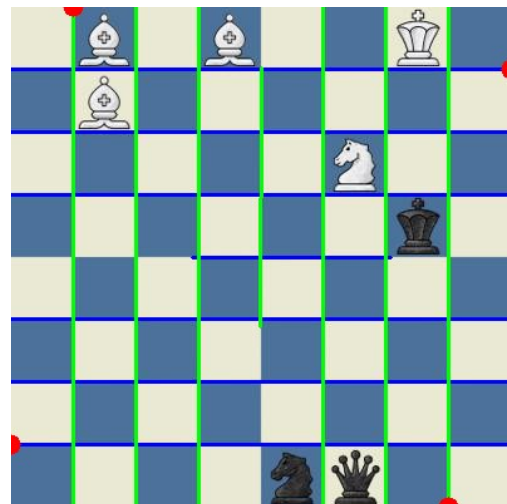


Figure 8: Hough lines + corners overlay

9.2 Classifier Confusion Matrix

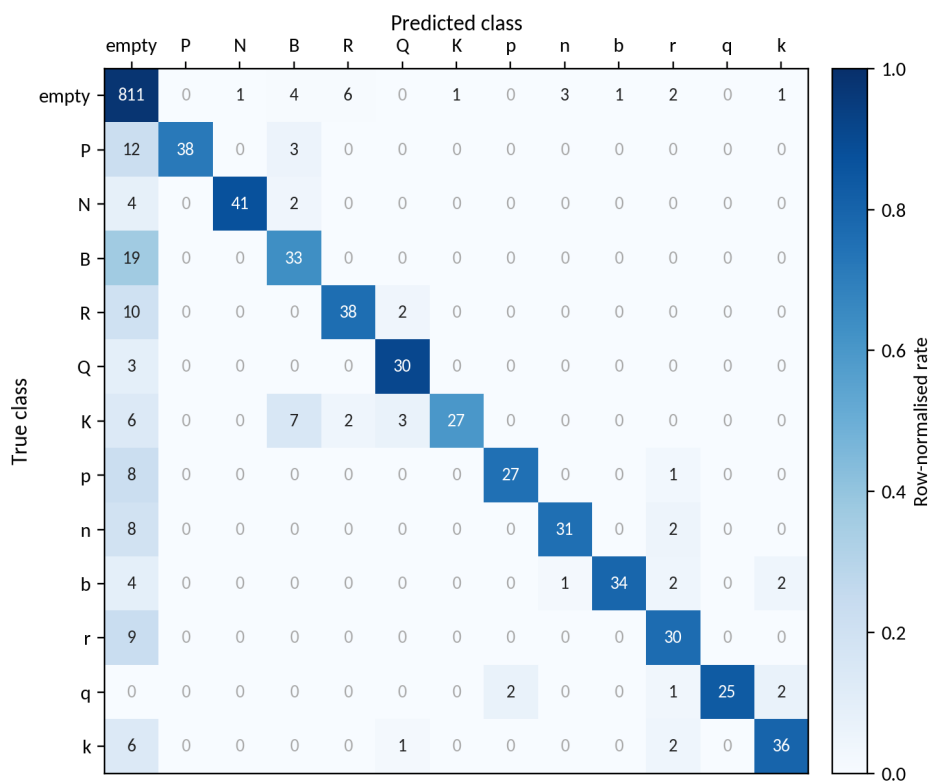


Figure 9: Confusion matrix on the combined held-out set (1,344 squares)

Most errors fall in the first column, i.e. pieces predicted as empty squares. The white bishop (19 of 52) and white pawn (12 of 53) rows are the worst affected. The white king is also confused with the bishop in 7 cases.

9.3 Measured Results

Table 5: Measured results

Metric	Value
Rectification success (15 synthetic boards)	100%, grid within 10 px
Corner error (mean / p95)	1.5 px / 2.0 px
Classifier per-square accuracy (synthetic held-out)	100% (640 squares)
Classifier per-square accuracy (combined held-out)	89.4% (1,344 squares)
Live demo, 5 rendered boards	53–60 / 64 squares each; 0 / 5 exact FEN
Full CLI time	≈ 0.08 s on CPU
Unit tests	16 / 16 pass

9.4 Preprocessing Ablation

Table 6: Preprocessing ablation on 15 synthetic boards

Setting	Success	Mean error (px)	p95 error (px)
CLAHE on + bilateral (shipped)	100%	1.5	2.0
CLAHE off + bilateral	100%	1.8	2.6
CLAHE on + Gaussian	100%	1.2	1.7
CLAHE off + Gaussian	100%	1.3	1.9

On clean synthetic boards every setting succeeds and the differences are at the level of sub-pixel noise; the Gaussian filter is even slightly ahead. The toggles were kept so that the same study can be repeated on real photos with lighting variation and wood grain, where CLAHE and the bilateral filter are expected to matter more. This is listed as future work.

10. Testing Approach

Table 7: Test suite

Test file	What it checks
test_rectify.py	Phase-1 acceptance on synthetic boards (grid tolerance 10 px, 90% pass bar).
test_squares.py	Slicer determinism and bias, FEN mapping, no-leakage data split, 5 MB model budget.
test_fen.py	Round-trips, orientation flip and tie handling, two-kings error, rejection of bad input.
test_pipeline.py	Exact-FEN smoke test on a composed board; engine degraded and live modes.

10.1 Acceptance Summary

- Detection: 15 / 15 boards rectified.
- Classifier: 89.4% on realistic data against a specification target of 95%. This gap is documented.
- End-to-end exact FEN: passes on synthetic positions, but 0 / 5 on the realistic sample. This is reported, not hidden.

11. Challenges Faced

Table 8: Challenges and resolutions

Challenge	Resolution
OpenCV 5 returns HoughLinesP as (N, 4) instead of (N, 1, 4).	Both shapes are handled.
Dashed board-border edges were missed by strict Hough settings.	Gap-tolerant tuning (minLineLength 60, maxLineGap 30).
The lattice hull locked onto interior grid lines.	Comparing against the contour result and keeping the larger quad fixed it.
Class-pure pseudo-photos gave a fake 100% accuracy.	Mixing classes in each photo exposed the true 77%; bolder shapes, mild augmentation and a wider network then recovered accuracy.
Train/serve skew (wood-tone training vs. grey boards).	Adding mixed realistic data closed most of the gap.
Laptop was too slow for training.	Training was moved to a Colab GPU using a notebook checked into the repository.
Streamlit data_editor in the deployed version has no help argument.	Replaced with a caption.

12. Learnings and Key Takeaways

- Classical detection generalises across board art styles for free, whereas the CNN only knows what it has been shown.
- Splitting by photo and using photos with mixed classes are what make the accuracy numbers believable.
- Exact-FEN accuracy is unforgiving: 89% per-square accuracy still gives 0 / 5 exact boards. This is why human-in-the-loop correction is part of the design.
- Returning a typed result from every stage turns each failure into evidence for the report instead of a crash.

13. Future Enhancements

- Retrain on real smartphone photos from the ChessReD dataset and repeat the preprocessing ablation under real lighting.
- Install Stockfish for live evaluation.
- Infer the side to move from move history instead of asking the user.
- Build a mobile capture app, and average over video frames for better robustness.

14. References

- [1] A. Masouris et al., “End-to-End Chess Recognition” and the ChessReD dataset, 4TU.ResearchData, DOI: 10.4121/99b5c721-280b-450b-b058-b2900b69a90f.
- [2] honi05, chess-positions-cv, Hugging Face Datasets (rendered boards with FEN file names).
- [3] Fenit annotated chessboard dataset, Mendeley Data, DOI: 10.17632/wpp9xnbcp6.
- [4] OpenCV documentation: Canny, HoughLinesP, getPerspectiveTransform.
- [5] python-chess, PyTorch and Streamlit documentation.